Level 2 Cache in Hibernate(EhCache)

What is Level 2 Cache?

- Level 2 (L2) cache is a **shared cache** across sessions.
- Unlike the **first-level cache** (which is session-specific), L2 cache is used by all sessions, improving performance for **multiple requests**.
- It helps reduce **database hits** by storing objects in memory after the first fetch.

Enabling Level 2 Cache in Hibernate 5:

1. **Add Dependencies in pom.xml**:

For Hibernate 5:

```
<dependency>
        <groupId>org.hibernate</groupId>
        <artifactId>hibernate-core</artifactId>
        <version>5.6.15.Final</version>
</dependency>

<dependency>
        <groupId>org.ehcache</groupId>
        <artifactId>ehcache</artifactId>
        <version>3.10.8</version>
</dependency>

<!-- https://mvnrepository.com/artifact/org.hibernate/hibernate-ehcache -->
<dependency>
   <groupId>org.hibernate</groupId>
   <artifactId>hibernate-ehcache</artifactId>
   <version>5.6.15.Final</version>
</dependency>
```

Note:

- As of time when preparing this document, Hibernate 6 is still under development for full support of second-level caching with Ehcache.
- If you encounter errors related to the Ehcache version in your pom.xml, consider downgrading Ehcache to version 3.8.1.

2. **Hibernate Configuration:**
    - Enable L2 cache by adding properties in the configuration file (hibernate.cfg.xml):

```
<property
name="hibernate.cache.region.factory_class">org.hibernate.cache.ehcache.EhCa
cheRegionFactory</property>
<property name="hibernate.cache.use_second_level_cache">true</property>
```

3. **Annotated Entity Class**:
    - Use annotations in your **entity class** to specify caching strategies:
    - **@Cacheable** *// Marks the entity as cacheable*
      @Cache(usage = CacheConcurrencyStrategy.READ_ONLY) *// Specifies the cache strategy*

```
import javax.persistence.Cacheable;
import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.Id;
import javax.persistence.Table;
import javax.persistence.Transient;
import org.hibernate.annotations.Cache;
import org.hibernate.annotations.CacheConcurrencyStrategy;
@Entity
@Table(name="StudentTable")
@Cacheable  // Marks the entity as cacheable
@Cache(usage = CacheConcurrencyStrategy.READ_ONLY)  // Specifies the
```

cache strategy

```
public class Student {
  @Id
  @Column(name="sid")
        private int id;

  @Column(name="sname")
        private String name;

  @Transient
        private String city;


        public Student() {
                System.out.println("Zero param constructor");
        }

        public int getId() {
                return id;
        }
        public void setId(int id) {
                this.id = id;
        }
        public String getName() {
                return name;
        }
        public void setName(String name) {
                this.name = name;
        }
        public String getCity() {
                return city;
        }
        public void setCity(String city) {
                this.city = city;
```

```
	}
	@Override
	public String toString() {
		return "Student [id=" + id + ", name=" + name + ", city=" + city +
"]";
	}

}
```

- @Cacheable: Marks the entity as cacheable, allowing Hibernate to store it in the second-level cache.
- @Cache(usage = CacheConcurrencyStrategy.READ_WRITE): Defines how the cache will behave:
  i. READ_ONLY: Used for entities that never change.
  ii. READ_WRITE: Used for entities that may be modified by the application.

Main.java

```
import org.hibernate.cfg.Configuration;
import jakarta.persistence.EntityExistsException;
import org.hibernate.HibernateException;
import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;
import org.hibernate.exception.ConstraintViolationException;
public class Main {
  public static void main(String[] args) {
    Configuration config = null;
    SessionFactory sessionFactory = null;
    Session session1 = null;
    Session session2=null;
```

```
Transaction transaction = null;
boolean flag = false;
try {
   config = new Configuration();
   config.configure();
   sessionFactory = config.buildSessionFactory();
   session1 = sessionFactory.openSession();

   Student s1=session1.get(Student.class, 12);
   Student s2=session1.get(Student.class, 12);
   System.out.println(s1);
   System.out.println(s2);

   session2=sessionFactory.openSession();
   Student s3=session2.get(Student.class, 12);
   Student s4=session2.get(Student.class, 12);
   System.out.println(s3);
   System.out.println(s4);

}  catch (HibernateException e) {
   System.out.println("Hibernate exception: " + e.getMessage());
} catch (Exception e) {
   System.out.println("Unexpected exception: " + e.getMessage());
} finally {

   if (session1 != null) {
      session1.close();
   }
   if (session2 != null) {
      session2.close();
   }
   if (sessionFactory != null) {
      sessionFactory.close();
   }
```

```
    }
  }
}
```

Output:

```
Hibernate:
    select
        student0_.sid as sid1_0_0_,
        student0_.sname as sname2_0_0_
    from
        StudentTable student0_
    where
        student0_.sid=?
Zero param constructor
Student [id=12, name=Shiva, city=null]
Student [id=12, name=Shiva, city=null]
Zero param constructor
Student [id=12, name=Shiva, city=null]
Student [id=12, name=Shiva, city=null]
```

Key Points to Remember:

- **Ehcache** is one of the most popular **second-level cache providers** for Hibernate.
- You need to ensure the version compatibility between Hibernate and Ehcache.
- **Hibernate 5** supports Ehcache easily, but for **Hibernate 6**, the support is still in progress.